

Tiny GPT in Pure Python

Detailed 16-Point Walkthrough: Data, Tokens, Autograd, Attention, Training, and Inference

Core idea. This file trains a tiny character-level GPT by repeatedly predicting the next character, measuring the error, sending gradients backward through every scalar operation, and updating the model weights until it can generate name-like text.

Original source. The code this walkthrough describes is `microgpt.py` by Andrej Karpathy — about 200 lines of pure Python, available at github.com/karpathy/microgpt.

How to use this document. Read the first three sections to understand the learning task, the tokens, and how much context the model can see. Then follow the model from forward pass to loss, backward pass, optimizer update, and inference. Code snippets are intentionally short and serve only as anchors to lines in the source file.

One-Page Orientation

Phase	Points	What to understand
Inputs and symbols	1–3	The model learns names as character sequences, with a 16-token context window.
Scalar engine	4–5	Every number remembers how it contributed to the final loss.
Model memory	6–7	The model’s knowledge lives in learned matrices inside <code>state_dict</code> .
Forward computation	8–10	The <code>gpt()</code> function turns the current token + past context into next-token probabilities.
Learning loop	11–12	Backpropagation computes gradients; Adam changes every weight.
Generation and takeaway	13–16	Inference samples one character at a time using the trained weights.

1. What is the model trying to learn?

The dataset is a list of names. Each name is treated as a short document. The model learns a simple but powerful task: **given the characters seen so far, predict the next character**.

For a name like `emma`, the code first wraps it with BOS markers to give `[BOS, e, m, m, a, BOS]`. The model then makes **five predictions**, one per position:

At position	Current input	Past context (from KV cache)	Should predict
0	BOS	(nothing)	e
1	e	BOS	m
2	m	BOS e	m
3	m	BOS e m	a
4	a	BOS e m m	BOS

Important: the model is **not** restarted with a longer prompt at each step. The five predictions happen inside one forward pass; previous tokens live in a growing key/value cache (see §9). The final BOS works like an end-of-name marker so the model learns where names stop.

The model is therefore not learning English grammar or whole words. It is learning statistical patterns of character sequences in names.

```
docs = [l.strip() for l in open("input.txt").read().strip().split("\n") if l.strip()]
```

2. What are the tokens?

The tokenizer is character-based. The code collects every unique character in the dataset and assigns each one a numeric ID. A special BOS token is added after the regular characters.

For the names dataset (lowercase a–z), the assignment is exact: a=0, b=1, c=2, ... , z=25, BOS=26. A name is converted into a list of numbers before the model sees it.

So emma → [BOS=26, e=4, m=12, m=12, a=0, BOS=26]. The model only ever sees token IDs, never the raw text.

```
uchars = sorted(set(''.join(docs)))
BOS = len(uchars)
vocab_size = len(uchars) + 1
```

3. How much context can the model see?

In transformer diagrams, **T** usually means *sequence length* — how many positions the model can attend to. In this code, T is bounded by the `block_size = 16` constant.

For one name like emma, the token sequence [BOS, e, m, m, a, BOS] has length 6, so the model makes 5 predictions ($n = \min(\text{block_size}, \text{len}(\text{tokens}) - 1) = 5$). For a longer name, the model would still cap out at `block_size` positions.

This implementation processes the sequence **one token at a time**, growing its key/value cache as it moves forward. It does not assemble the whole T-by-dimension matrix at once. (That’s what a full GPU implementation would do; this one trades speed for clarity.)

Term	Meaning in this code
<code>block_size = 16</code>	Maximum context length, in token positions
Token	One character-level symbol, or BOS
Context length	Maximum number of token positions the model can use
“16 characters”	Acceptable shorthand, but slightly imprecise — BOS counts as a token

Learning line: this model has a 16-**token** context length. Because it is character-level, that roughly means 16 character positions, but BOS also counts as a token.

```
block_size = 16
n = min(block_size, len(tokens) - 1)
```

4. What is Value?

`Value` is the tiny scalar object that makes autograd possible. Each `Value` stores one number and one gradient.

- `value.data` is the actual number used during the forward pass.
- `value.grad` is the derivative of the final loss with respect to that number. In plain English: how much the loss would change if this number changed slightly.

Every model weight is a `Value` object. A single weight might conceptually look like `Value(data=0.037, grad=-0.004)`.

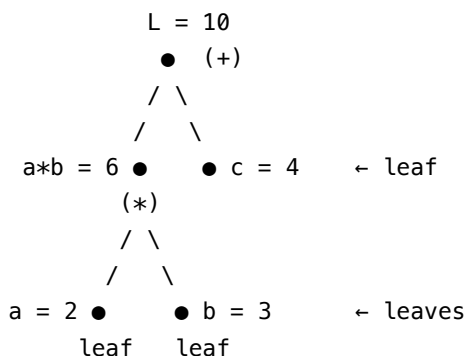
```
class Value:
    self.data = data
    self.grad = 0
```

5. What does the computation graph do?

Every mathematical operation creates a **new** `Value`. That new `Value` remembers two things: which earlier `Values` produced it, and the **local derivative** of the operation with respect to each input.

For example, if $c = a * b$, then c remembers it came from a and b , and that $dc/da = b$ and $dc/db = a$. This is enough to send gradients backward later.

$$L = a*b + c \quad \text{with } a=2, b=3, c=4 \quad \rightarrow \quad L = 10$$



Each non-leaf node remembers:

- its data (forward value)
- its children (the inputs that produced it)
- the local derivative w.r.t. each child

Note on terminology. In this code a `Value`'s `_children` are its **inputs** — the parents in the math sense. When `backward()` walks the graph, it pushes gradient **from a node down to its `_children`** (its inputs). If you're used to "child = downstream consumer", flip the picture in your head before reading `backward()`.

When `loss.backward()` is called, the graph is walked in reverse topological order and every input receives its share of the gradient via the chain rule. This is a tiny version of what PyTorch autograd does.

```
def __mul__(self, other):
    return Value(self.data * other.data, (self, other), (other.data, self.data))

loss.backward()
```

6. Where is the model's memory or knowledge?

The model's knowledge is stored in `state_dict`. This dictionary contains all the trainable matrices: token embeddings, position embeddings, attention weights, MLP weights, and the output head.

The model has no other stored knowledge. The patterns it learns are distributed across thousands of scalar `Value` objects inside these matrices.

Training changes the `.data` field of these `Value` objects. **That is what learning means in this file.**

```
state_dict = {'wte': ..., 'wpe': ..., 'lm_head': ...}
state_dict[f'layer{i}.attn_wq']
state_dict[f'layer{i}.mlp_fc1']
```

7. What are the main learned matrices?

The compressed names in `state_dict` come from GPT-2 source code. Here's the decoder:

Code name	Stands for	What it is
<code>wte</code>	W ord T oken E mbedding	Per-character learned vector ($\text{vocab} \times \text{n_emdb}$)
<code>wpe</code>	W ord P osition E mbedding	Per-position learned vector ($\text{block} \times \text{n_emdb}$)
<code>attn_wq</code>	Attention W eight: Q uery	Per-layer query projection
<code>attn_wk</code>	Attention W eight: K ey	Per-layer key projection
<code>attn_wv</code>	Attention W eight: V alue	Per-layer value projection
<code>attn_wo</code>	Attention W eight: O utput	Mixes attention heads back together
<code>mlp_fc1</code>	MLP F ully- C onnected 1	Expands the representation
<code>mlp_fc2</code>	MLP F ully- C onnected 2	Contracts it back
<code>lm_head</code>	L anguage- M odel head	Converts the final vector to vocab scores

`wte` gives each character a learned vector of 16 numbers; `wpe` gives each possible sequence position its own learned vector of 16 numbers.

Important: `wte` and `wpe` are not calculated from the character or position using a fixed formula. They are **learned lookup tables**. At the beginning, they are filled with small random numbers. During training, backpropagation and Adam gradually change those numbers so they become useful for predicting the next character.

`wte` answers: *what token is this?* `wpe` answers: *where is this token?* The model adds them together so the input vector carries both identity and position.

For example, in the name `emma`, the first `m` (at position 2) and the second `m` (at position 3) share the same token vector `wte[m]`, but they use different position vectors `wpe[2]` and `wpe[3]`. That is how the model can distinguish them.

The attention matrices `attn_wq` / `wk` / `wv` / `wo` learn the query (what is the current token looking for?), the key (what does each previous token offer?), the value (what does it contribute?), and the output projection (how to mix multi-head outputs back together).

The MLP matrices `mlp_fc1` and `mlp_fc2` reshape and refine the representation after attention. `lm_head` converts the final hidden vector into scores for the next character.

```
state_dict["wte"] = matrix(vocab_size, n_embd)           # [27, 16]
state_dict["wpe"] = matrix(block_size, n_embd)           # [16, 16]
tok_emb = state_dict["wte"][token_id]
pos_emb = state_dict["wpe"][pos_id]
x = [t + p for t, p in zip(tok_emb, pos_emb)]
```

8. What happens during one forward pass?

The forward pass is the function `gpt(token_id, pos_id, keys, values)`. It receives the current token, the current position, and the accumulated keys and values from previous tokens.

The model performs these steps in order:

1. **Look up** the token and position embeddings, add them together.
2. **Normalise** the result with `rmsnorm` — divide every entry by the root-mean-square of the vector. This stabilises the scale of activations between layers so the next operation sees inputs in a predictable range.
3. **Attention** sub-block: norm again, then attention, then add the pre-attention input back (the *residual* connection — it lets gradients flow cleanly from the loss back to the earliest layers).
4. **MLP** sub-block: norm again, then two Linear layers with a ReLU between, then add the pre-MLP input back (another residual).
5. **Output projection** (`lm_head`) — convert the 16-dim hidden vector into 27 numbers, one per vocabulary token. These are the **logits**.

The output is not yet a probability. It is a list of raw scores, one score per possible next token.

```
def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict["wte"][token_id]
    pos_emb = state_dict["wpe"][pos_id]
    x = [t + p for t, p in zip(tok_emb, pos_emb)]
    x = rmsnorm(x)                                     # stabilise scale
    # ... attention block (with norm + residual) ...
    # ... MLP block (with norm + residual) ...
    logits = linear(x, state_dict["lm_head"])
```

9. What does attention do here?

At each position the model creates **q**, **k**, **v** from the current hidden vector. The current position's **k** and **v** are then appended to the cache so they are available for this and all future positions.

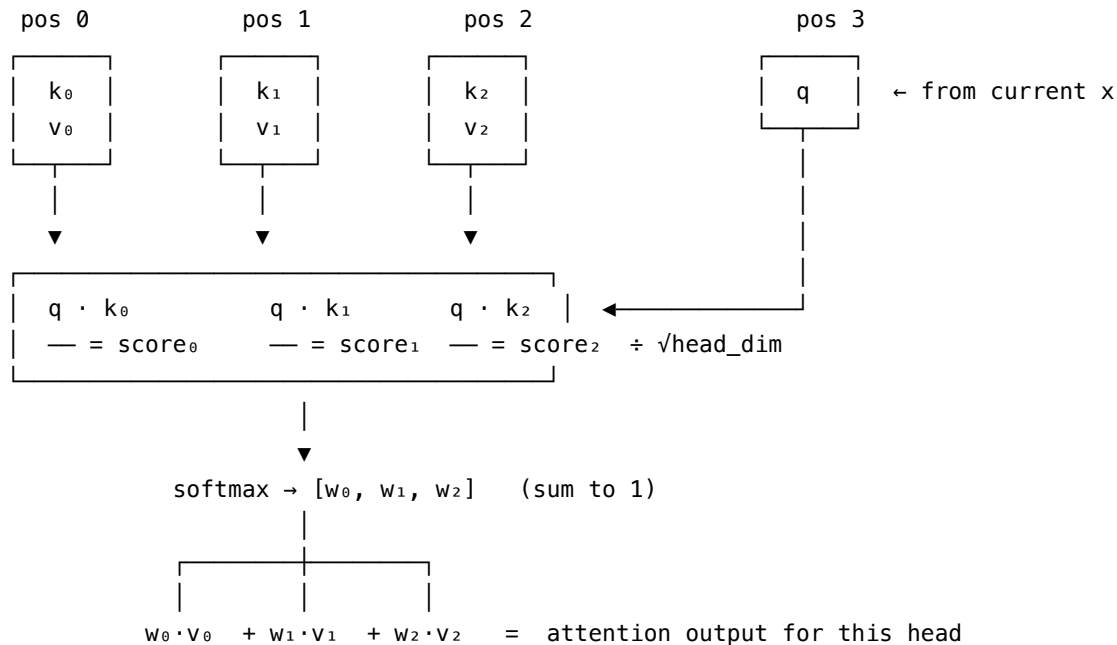
For each attention head, the current **query** is compared with every **key already in the cache** (one comparison per past timestep, including the current one). Each comparison is a dot product, divided by the square root of the head size — this prevents the softmax from saturating as head size grows.

The results are called *attention logits*, and softmax turns them into a probability distribution over past positions.

The output of one head is the **weighted sum of the cached values**, using the softmax weights. In plain English: attention decides how much each earlier token should influence the current prediction.

Past tokens already in the cache:

Current position:



Multi-head: the 16-dim embedding is split into 4 slices of 4 dims each. Attention is computed **4 times independently**, once per slice; the 4 head outputs are then concatenated back to length 16 and projected once more through `attn_w0`. Different heads can learn to attend to different things — one might learn “look at the last letter”, another might learn “look at BOS”.

```
q = linear(x, state_dict[f"layer{li}.attn_wq"])
k = linear(x, state_dict[f"layer{li}.attn_wk"])
v = linear(x, state_dict[f"layer{li}.attn_wv"])
attn_weights = softmax(attn_logits) # one logit per past timestep
```

10. What are logits, probabilities, and loss?

Logits are the raw scores produced by `lm_head` — one number per possible next token. **Softmax** converts these scores into probabilities that add up to 1.

The **loss** checks the probability assigned to the **correct** next token. If the model gives a high probability to the correct token, the loss is low. If it gives a low probability, the loss is high.

The line `-probs[target_id].log()` is the negative log-likelihood for one prediction. The document loss is the **average** of these per-position losses.

```
logits = gpt(token_id, pos_id, keys, values)
probs = softmax(logits)
```

```
loss_t = -probs[target_id].log()
loss = (1 / n) * sum(losses)
```

11. What happens during training?

Each training step picks one document, converts it to tokens, predicts the next character at every position, calculates the average loss, runs backpropagation, updates the parameters, and resets gradients to zero.

- The **forward pass** builds the computation graph all the way to the loss.
- The **backward pass** fills in gradients for every parameter.
- The **optimizer** uses those gradients to change the parameter values.

Common misunderstanding. Many people assume training only fine-tunes the embeddings. That is only partly true. Training tunes **every** trainable weight: **wte**, **wpe**, attention weights, MLP weights, and **lm_head**.

What each parameter group is learning:

Parameter group	What it learns
wte	What each character/token means in a useful numerical form
wpe	What each position means in a useful numerical form
attn_wq, attn_wk, attn_wv, attn_wo	How tokens should look at previous tokens
mlp_fc1, mlp_fc2	How to transform and combine patterns after attention
lm_head	How to convert the final internal vector into next-character scores

The better statement is: training adjusts all stored numbers in the model so that, given the previous characters, the model assigns higher probability to the correct next character.

Training is therefore a repeated cycle: predict, measure error, compute gradients, update weights, and try again.

```
for step in range(num_steps):
    loss.backward()
    p.data -= ...
    p.grad = 0
```

12. What does Adam do?

Adam is the optimizer. It decides how each parameter should move based on its gradient and its recent gradient history.

- **m** is a moving average of the gradient itself — gives the update *direction* some memory.
- **v** is a moving average of the **squared** gradient — gives each parameter its own effective learning rate.

These buffers make the update smoother and better scaled than a raw gradient step. The bias-corrected m_hat and v_hat versions undo the fact that m and v start at zero (which would otherwise bias the early steps toward zero).

The learning rate also **decays linearly** to zero over training ($lr_t = lr * (1 - step / num_steps)$), so updates get smaller as the model fine-tunes.

The actual learning happens on the `p.data -= ...` line. The code then clears `p.grad` so the next training step starts fresh.

```
m[i] = beta1 * m[i] + (1 - beta1) * p.grad
v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
p.grad = 0
```

13. What happens during inference?

Inference starts with BOS. The model predicts probabilities for the next character, samples one token, appends it to the generated name, and feeds that token back into the model.

This repeats until the model samples BOS again or reaches the maximum block size. The second BOS acts as the end marker.

The generated name is not copied directly from the training set. It is produced one character at a time from the statistical patterns stored in the weights.

Temperature controls the sampling style. Before softmax, the logits are divided by a constant temperature (here 0.5):

- temperature < 1 → sharper distribution. Conservative, repetitive.
- temperature = 1 → sample directly from the model's distribution.
- temperature > 1 → flatter distribution. Creative, possibly silly.

```
token_id = BOS
probs = softmax([l / temperature for l in logits])
token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
if token_id == BOS: break
```

14. The whole algorithm in one sentence

This file trains a tiny character-level GPT by creating thousands of scalar weights, using those weights to predict the next character in names, calculating how wrong the predictions are, pushing gradients backward through every scalar operation, and updating the weights until the model can generate name-like sequences.

That sentence captures the full loop: weights create predictions, predictions create loss, loss creates gradients, gradients update weights, and updated weights create better predictions.

15. The clean mental model

The entire system can be understood as **six parts**: data, tokenizer, parameters, forward pass, backward pass, and optimizer.

- Data provides names.
- The tokenizer converts names into numbers.
- Parameters store the model knowledge.
- The forward pass predicts the next token.
- The backward pass calculates how every weight contributed to the error.
- The optimizer adjusts the weights.

Once these six parts are clear, the full code becomes much easier to read.

16. Most important learning point

The heart of training is `loss.backward()`. This computes how the final loss depends on every trainable `Value` object.

The heart of learning is the parameter update: `p.data -= something based on the gradient`. This is where the model changes.

Together they mean: first find out how every weight contributed to the error, then slightly change every weight so the same mistake is less likely next time.

The cleanest learning line: **embeddings give the model its starting representation; training tunes the embeddings AND every later transformation so the final prediction becomes better.**

```
loss.backward()
p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
```

Final Recap: Read the Code in This Order

Order	Code area	Question to ask
1	<code>docs</code> , <code>uchars</code> , <code>BOS</code> , <code>block_size</code>	What is the training task, how is text represented, and how much context can the model see?
2	<code>Value</code>	How does each scalar remember data and gradient?
3	<code>state_dict</code>	Where are all trainable weights stored?
4	<code>gpt()</code>	How does one token become next-token logits? Watch for <code>rmsnorm</code> , residuals, and the per-head attention loop.
5	<code>softmax</code> and <code>loss</code>	How is wrongness measured?
6	<code>loss.backward()</code>	How are gradients computed?
7	Adam update	How do weights change? (Note the linear LR decay.)
8	inference loop	How does the trained model generate names? (Note the temperature.)
